

Progetto Sistemi Operativi

Booking System

Daniele Natalini

Matricola: 7053584

`daniele.natalini@edu.unifi.it`

Corso di Laurea in Informatica, Università degli Studi di Firenze

A.A. 2025/26

1 Introduzione

Il progetto di laboratorio di Sistemi Operativi richiedeva l'implementazione di un Booking System per la prenotazione di risorse condivise, basato su un'architettura client-server e realizzato utilizzando socket TCP nel dominio `AF_INET`.

Il sistema consente a più client di collegarsi ad un server centrale e prevede due tipologie di utenti: utenti standard e amministratori. Gli utenti standard possono creare nuove richieste di prenotazione e consultare esclusivamente quelle associate al proprio account. Gli amministratori possono invece visualizzare tutte le prenotazioni, approvarle, rifiutarle e modificarne lo stato.

Il controllo dei conflitti viene effettuato lato server: due prenotazioni relative alla stessa risorsa non possono sovrapporsi temporalmente. Poiché il server può gestire più richieste contemporaneamente, è stato inoltre implementato un meccanismo di sincronizzazione per garantire la consistenza dell'archivio anche in presenza di accessi concorrenti.

2 Architettura generale del sistema

L'architettura del sistema è di tipo client-server.

- Il **server** è un processo persistente che rimane in ascolto su una socket TCP e gestisce autenticazione, registrazione degli utenti, prenotazioni e accesso ai file di persistenza.
- I **client** sono processi indipendenti che si collegano al server e interagiscono con il sistema attraverso un menu testuale differenziato in base al ruolo dell'utente autenticato.

Il server può gestire più client contemporaneamente. Ad ogni nuova connessione accettata viene creato un processo figlio tramite `fork()`. Il processo padre continua ad accettare nuove connessioni, mentre il processo figlio gestisce la comunicazione con il client ad esso associato.

Scelta progettuale:

Per evitare la permanenza di processi zombie, il server gestisce il segnale `SIGCHLD`. Alla terminazione di uno o più processi figli viene utilizzata `waitpid()` con l'opzione `WNOHANG`, in modo da raccogliere i figli terminati senza bloccare il server.

3 Scelte di comunicazione e IPC

3.1 Utilizzo delle socket `AF_INET`

La comunicazione tra client e server è implementata mediante socket TCP nel dominio `AF_INET`. Il server rimane in ascolto sull'indirizzo locale `127.0.0.1` e sulla porta `54321`.

Nel file `server.c` vengono utilizzate principalmente le chiamate di sistema `socket()`, `bind()`, `listen()` e `accept()`.

Lato client, nel file `client.c`, viene utilizzata `connect()` per stabilire la connessione con il server.

Dopo ogni `accept()`, il server utilizza `fork()` per creare un processo dedicato al nuovo client. Il processo figlio chiude la socket di ascolto e mantiene la socket relativa al client, mentre il processo padre chiude la socket del client e torna ad attendere nuove connessioni.

Questa organizzazione permette a più utenti di utilizzare il Booking System contemporaneamente.

4 Struttura del codice

Il progetto è stato suddiviso in più moduli con responsabilità differenti:

- **server.c**: gestione delle connessioni, delle sessioni e interpretazione delle richieste dei client;
- **client.c**: connessione al server e gestione dei menu interattivi;
- **archive.c** e **archive.h**: gestione dell'archivio delle prenotazioni, controllo dei conflitti e sincronizzazione degli accessi;
- **users.c** e **users.h**: autenticazione, registrazione e gestione degli utenti;
- **strutil.c** e **strutil.h**: funzioni di supporto per la validazione di date e orari;
- **common.h**: definizioni condivise tra client e server, tra cui la struttura delle prenotazioni, i ruoli e gli stati;
- **Makefile**: compilazione del progetto.

5 Gestione degli utenti e autenticazione

Il sistema prevede due categorie di utenti:

- **Utenti standard**, che possono creare nuove prenotazioni e consultare le proprie richieste;
- **Amministratori**, che possono visualizzare tutte le prenotazioni e modificarne lo stato.

Gli account vengono memorizzati nel file testuale **utenti.dat**, utilizzando il formato:

```
username;password;ruolo
```

dove il ruolo 0 identifica un utente standard e il ruolo 1 un amministratore.

L'autenticazione viene effettuata lato server. Il client invia le credenziali attraverso il comando **LOGIN**; dopo una verifica corretta, il server associa alla connessione lo username e il ruolo dell'utente.

Il sistema permette inoltre la registrazione di nuovi utenti standard tramite il comando **REGISTER**. Prima dell'inserimento viene verificato che lo username non sia già presente.

Scelta progettuale:

I controlli di autorizzazione vengono sempre effettuati lato server. Il menu del client nasconde le operazioni non disponibili per il ruolo corrente, ma questo non costituisce il meccanismo di sicurezza. Anche una richiesta costruita manualmente viene quindi rifiutata se l'utente non possiede i permessi necessari.

6 Gestione delle prenotazioni

6.1 Struttura dati della prenotazione

Ogni prenotazione è rappresentata tramite una struttura **Booking**, contenente:

- identificativo univoco;
- risorsa;
- data;
- ora di inizio;
- ora di fine;
- richiedente;
- stato.

Gli stati disponibili sono **pending**, **approved** e **rejected**, corrispondenti rispettivamente a prenotazione in attesa, approvata e rifiutata.

Una nuova prenotazione viene inizialmente inserita con stato **pending**; sarà successivamente l'amministratore a poterla approvare o rifiutare.

6.2 Persistenza su file

Le prenotazioni vengono memorizzate nel file testuale `prenotazioni.dat`, una per riga, secondo il formato:

```
id;risorsa;data;ora_inizio;ora_fine;richiedente;stato
```

Il modulo `archive.c` gestisce il caricamento e il salvataggio dell'archivio, la generazione degli identificativi e il controllo delle sovrapposizioni.

La persistenza su file permette di conservare utenti e prenotazioni anche dopo la terminazione del server.

6.3 Sincronizzazione degli accessi

Poiché ogni client viene gestito da un processo figlio differente, più processi possono tentare contemporaneamente di leggere o modificare lo stesso archivio.

Senza sincronizzazione, due processi potrebbero ad esempio verificare nello stesso momento la disponibilità della medesima risorsa e inserire due prenotazioni incompatibili.

Per evitare questa race condition è stato introdotto un file di lock. L'acquisizione avviene tramite:

```
open(LOCK_FILE, O_CREAT | O_EXCL, 0644);
```

La combinazione `O_CREAT | O_EXCL` permette ad un solo processo di creare il file. Gli altri processi devono attendere il rilascio del lock prima di accedere alla sezione critica.

Scelta progettuale:

Il lock protegge l'intera sequenza di caricamento dell'archivio, controllo dei conflitti ed eventuale scrittura. In questo modo un altro processo non può modificare i dati tra il controllo della disponibilità e il salvataggio della prenotazione.

Al termine dell'operazione il lock viene rilasciato tramite `unlink()`. È inoltre previsto un controllo per eliminare un eventuale lock obsoleto rimasto presente in seguito ad una terminazione anomala.

7 Protocollo di comunicazione

La comunicazione utilizza un protocollo testuale basato su comandi. Ogni richiesta contiene il nome dell'operazione seguito dagli eventuali parametri, separati tramite il carattere “;”.

Alcuni esempi sono:

```
LOGIN;utente;password
REGISTER;utente;password
NEW;room1;15/09/2026;09:00;11:00
LIST_MINE
SEARCH_MINE;room1;-;pending
LIST_ALL
APPROVE;1
REJECT;2
SET_STATUS;3;pending
```

Il server interpreta il comando, controlla autenticazione e permessi, valida gli eventuali parametri ed esegue l'operazione.

Le risposte indicano l'esito tramite OK oppure ERR. In caso di errore viene aggiunto un codice che ne identifica la categoria, ad esempio AUTH, SYNTAX, CONFLICT o NOTFOUND.

Scelta progettuale:

Per le risposte del server viene utilizzato un semplice meccanismo di framing: viene inviata prima la lunghezza della risposta e successivamente il contenuto. Il client continua a leggere fino a quando non ha ricevuto il numero di byte dichiarato.

Questo permette di gestire correttamente anche risposte più lunghe, come l'elenco delle prenotazioni.

8 Implementazione delle funzionalità del sistema

8.1 Creazione di una prenotazione

Un utente standard autenticato può creare una nuova prenotazione specificando risorsa, data, ora di inizio e ora di fine.

Prima del salvataggio il server:

- verifica l'autenticazione;
- verifica che la risorsa sia nel formato Room1–Room10;
- verifica che l'orario richiesto sia compreso tra le 08:00 e le 20:00;
- controlla la validità della data e degli orari;
- verifica che la data non appartenga al passato;
- verifica che l'ora di inizio preceda quella di fine;
- acquisisce il lock;
- verifica l'assenza di conflitti;
- assegna un nuovo identificativo;
- salva la prenotazione con stato **pending**;
- rilascia il lock.

Scelta progettuale:

Il confronto tra risorse utilizzato per i conflitti non distingue tra maiuscole e minuscole: al momento del salvataggio, la risorsa viene comunque riscritta in forma canonica (Room1), così che **room1**, **ROOM1** e **Room1** vengano tutte salvate e mostrate in modo identico e coerente.

Scelta progettuale:

Per semplicità le risorse prenotabili sono limitate a un insieme fisso di dieci aule, denominate Room1–Room10: qualunque altro nome viene rifiutato con un errore di sintassi. Analogamente, è stato introdotto un orario di apertura (08:00–20:00) entro cui devono rientrare sia l'ora di inizio sia l'ora di fine di ogni prenotazione. Questo vincolo evita inoltre, come effetto collaterale utile, la necessità di gestire prenotazioni che attraversano la mezzanotte (ad esempio dalle 22:00 alle 00:00), un caso che avrebbe richiesto una gestione separata nel confronto tra gli orari.

```

--- Standard User Menu ---
1) New booking
2) View all my bookings
3) Search my bookings by field
4) Exit
Choice: 1
Resource (Room1 to Room10): Room1
Date (DD/MM/YYYY): 20/09/2026
Start time (HH:MM): 09:00
End time (HH:MM): 10:00
OK;id=1 status=pending

```

Figura 1: Creazione di una nuova prenotazione

8.2 Consultazione delle prenotazioni

Gli utenti standard possono visualizzare le proprie prenotazioni tramite **LIST_MINE** oppure effettuare una ricerca per campi tramite **SEARCH_MINE**.

La ricerca permette di utilizzare risorsa, data e stato come filtri. Il simbolo “-” indica che un determinato campo non deve essere utilizzato come filtro.

Indipendentemente dai parametri ricevuti, il server seleziona solamente le prenotazioni il cui richiedente corrisponde allo username autenticato.

Scelta progettuale:

Il controllo sul proprietario della prenotazione viene effettuato lato server e non dipende dai dati inviati dal client. In questo modo ogni utente può consultare esclusivamente le proprie richieste, garantendo la riservatezza richiesta dal sistema.

```
--- Standard User Menu ---
1) New booking
2) View all my bookings
3) Search my bookings by field
4) Exit
Choice: 2
ID   Resource      Date      Start    End      Requester  Status
-----
1    Room1          20/09/2026 09:00    10:00    daniele.natalini pending
```

Figura 2: Consultazione delle proprie prenotazioni

8.3 Controllo dei conflitti

Due prenotazioni sono considerate in conflitto quando riguardano la stessa risorsa, nella stessa data, e i rispettivi intervalli temporali si sovrappongono.

Il controllo della sovrapposizione può essere espresso come:

$$\text{inizio1} < \text{fine2} \quad \text{e} \quad \text{inizio2} < \text{fine1}$$

Gli orari vengono convertiti in minuti per semplificare il confronto.

Le prenotazioni con stato **rejected** non occupano la risorsa e non vengono quindi considerate nel controllo.

Se viene individuata una sovrapposizione, il server rifiuta la nuova richiesta restituendo un errore **CONFLICT**.

```
--- Standard User Menu ---
1) New booking
2) View all my bookings
3) Search my bookings by field
4) Exit
Choice: 1
Resource (Room1 to Room10): Room1
Date (DD/MM/YYYY): 20/09/2026
Start time (HH:MM): 09:30
End time (HH:MM): 10:30
ERR;CONFLICT;A conflicting booking already exists for this resource
```

Figura 3: Rilevamento di un conflitto tra prenotazioni

8.4 Gestione delle prenotazioni da amministratore

L'amministratore dispone di operazioni aggiuntive rispetto all'utente standard.

Può visualizzare l'intero archivio tramite **LIST_ALL** e può modificare lo stato delle prenotazioni tramite:

- **APPROVE;**
- **REJECT;**
- **SET_STATUS.**

Prima di ogni operazione il server verifica che la sessione appartenga effettivamente ad un amministratore.

Prima di approvare una prenotazione viene inoltre effettuato nuovamente il controllo dei conflitti. Questo evita che una prenotazione precedentemente rifiutata possa essere successivamente approvata se, nel frattempo, la stessa risorsa è stata occupata da un'altra prenotazione incompatibile.

```

--- Administrator Menu ---
1) View all bookings
2) Approve a booking
3) Reject a booking
4) Manually change a booking's status
5) Exit
Choice: 1
ID   Resource   Date       Start    End      Requester  Status
-----
1    Room1       20/09/2026 09:00    10:00    daniele.natalini pending

```

Figura 4: Visualizzazione delle prenotazioni da amministratore

```

--- Administrator Menu ---
1) View all bookings
2) Approve a booking
3) Reject a booking
4) Manually change a booking's status
5) Exit
Choice: 2
Booking ID: 1
OK;id=1 newStatus=approved

```

Figura 5: Modifica dello stato di una prenotazione

9 Compilazione del progetto

Per la compilazione è stato utilizzato un `Makefile` posto nella directory principale del progetto.

Il comando:

```
make
```

compila server e client.

È inoltre possibile compilare separatamente i due programmi tramite `make server` e `make client`.

Il comando:

```
make clean
```

rimuove gli eseguibili e gli eventuali file oggetto generati durante la compilazione.

10 Esecuzione del programma

L'esecuzione del sistema prevede innanzitutto l'avvio del server:

```
./server
```

Il server rimane quindi in ascolto sull'indirizzo `127.0.0.1:54321`.

Successivamente è possibile avviare uno o più client in terminali separati:

```
./client
```

Ogni client si connette al server e permette inizialmente di effettuare il login oppure di registrare un nuovo utente standard. Dopo l'autenticazione viene mostrato il menu corrispondente al ruolo dell'utente.

Per effettuare l'accesso come amministratore deve essere presente nel file `utenti.dat` un account configurato con ruolo `1`.

11 Gestione degli errori e casi critici

Sono stati considerati diversi casi di errore, tra cui:

- credenziali di login errate;
- registrazione di uno username già esistente;
- tentativi di accesso non autorizzato;
- comandi con sintassi non valida;
- date o orari non validi;
- inserimento di date appartenenti al passato;
- ora di inizio maggiore o uguale all'ora di fine;
- prenotazioni in conflitto;
- ricerca di prenotazioni inesistenti;
- errori nell'accesso ai file;
- problemi nell'acquisizione del lock;
- disconnessione del client.

In presenza di un errore, il server restituisce una risposta appropriata senza compromettere la stabilità del sistema.

La combinazione dei controlli lato server, del meccanismo di lock e della separazione dei permessi permette inoltre di mantenere la consistenza dell'archivio e di garantire che ciascun utente standard possa accedere esclusivamente alle proprie prenotazioni.

12 Limiti noti

Vengono qui documentati esplicitamente alcuni limiti individuati in fase di sviluppo, per i quali si è scelto di non introdurre soluzioni sproporzionate rispetto allo scopo del progetto:

- il server legge ogni comando in arrivo dal client con una singola `read()`, assumendo che l'intero comando venga ricevuto in un'unica chiamata; su connessione locale e con comandi brevi come i nostri il rischio di frammentazione è trascurabile in pratica, ma non è escluso in linea di principio da TCP;
- il meccanismo di recupero del lock obsoleto riduce ma non elimina teoricamente una finestra di race condition nel caso limite in cui due processi decidano quasi contemporaneamente che il medesimo lock è da considerarsi orfano, subito dopo una terminazione anomala del server: uno scenario estremamente raro, la cui completa eliminazione richiederebbe primitive di sincronizzazione più avanzate non trattate nel corso;
- le password sono memorizzate e trasmesse in chiaro: in un contesto reale andrebbero sottoposte a hashing prima del salvataggio.